ASSIGNMENT NO : 11

🔷 PART 1: CONNECT & SETUP

✅ Step 1: Connect to server

ssh hadoop@10.10.15.81

✅ Step 2: Check files

ls

✅ Step 3: Go to Hadoop folder

cd $HADOOP_HOME/sbin

Moves to Hadoop system scripts folder.

- `$HADOOP_HOME` = Hadoop installation path
- `sbin` = contains start/stop scripts

✅ Step 4: Check scripts

ls

You will see scripts like:

- `start-dfs.sh`
- `start-yarn.sh`

🔷 PART 2: CREATE INPUT DATA

✅ Step 5: Create your working folder

mkdir 31415

cd 31415

Create directory using your roll number.

✅ Step 6: Create input file

nano data.txt

Enter text like:

hello world hello hadoop

big data hadoop world

Save:

- `Ctrl + O` → Save
- `Enter`
- `Ctrl + X` → Exit

🔷 PART 3: WRITE JAVA CODE

✅ Step 7: Create Java file

nano WordCount.java

🔷 PART 4: COMPILE & CREATE JAR

✅ Step 8: Compile Java

javac -classpath `hadoop classpath` WordCount.java

👉 What happens:

- Compiles Java → `.class` files
- Uses Hadoop libraries

If error:

javac -source 1.8 -target 1.8 -classpath `hadoop classpath` WordCount.java

✅ Create JAR file

jar cf WordCount.jar WordCount*.class

👉 Combines `.class` files into executable JAR

🔷 PART 5: START HADOOP

✅ Step 9: Start services

start-dfs.sh

👉 Starts HDFS (storage system)

start-yarn.sh

👉 Starts YARN (resource manager)

✅ Check running processes

jps

👉 You should see:

- NameNode
- DataNode
- ResourceManager
- NodeManager

🔷 PART 6: HDFS OPERATIONS

✅ Step 10A: Create HDFS root

hdfs mkdir hdfs://localhost:9000/

Remove old directory (IMPORTANT)

hdfs rm -rf hdfs://localhost:9000/31415

👉 Deletes old output (Hadoop won't overwrite)

✅ Create your folder

hdfs mkdir hdfs://localhost:9000/31415

✅ Create input folder

hdfs mkdir hdfs://localhost:9000/31415/input

🔷 PART 7: UPLOAD FILE

✅ Step 11: Upload data

hdfs put data.txt hdfs://localhost:9000/31415/input

👉 Copies local file → HDFS

🔷 PART 8: RUN MAPREDUCE JOB

✅ Step 12: Run job

hadoop jar WordCount.jar WordCount /31415/input /31415/output

👉 Arguments:

- `/input` → input folder
- `/output` → output folder

⚙️ WHAT HAPPENS INTERNALLY (VERY IMPORTANT 🔥)

1. File split into blocks (e.g., 128MB)
2. Each block → Mapper runs
3. Mapper outputs (word,1)
4. Shuffle & Sort groups keys
5. Reducer sums values
6. Output stored in HDFS

🔷 PART 9: CHECK OUTPUT

✅ Step 13: List output

hdfs ls hdfs://localhost:9000/31415/output

👉 You'll see:

part-r-00000

✅ Step 14: View result

hdfs cat hdfs://localhost:9000/31415/output/part-r-00000

👉 Output example:

big 1

data 1

hadoop 2

hello 2

world 2

🔷 PART 10: EXIT

exit

CODE:

```
import java.io.IOException;
import java.util.StringTokenizer;

import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;

import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;

import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class WordCount {

   // 🔹 Mapper Class
   public static class TokenizerMapper
         extends Mapper<Object, Text, Text, IntWritable> {

      private final static IntWritable one = new IntWritable(1);
      private Text word = new Text();

      public void map(Object key, Text value, Context context)
            throws IOException, InterruptedException {
```

```
        StringTokenizer itr = new StringTokenizer(value.toString());

        while (itr.hasMoreTokens()) {
            word.set(itr.nextToken());
            context.write(word, one);
        }
    }
}

// 🔹 Reducer Class
public static class IntSumReducer
        extends Reducer<Text, IntWritable, Text, IntWritable> {

    private IntWritable result = new IntWritable();

    public void reduce(Text key, Iterable<IntWritable> values,
                       Context context)
            throws IOException, InterruptedException {

        int sum = 0;

        for (IntWritable val : values) {
            sum += val.get();
        }

        result.set(sum);
        context.write(key, result);
    }
}

// 🔹 Main Method
```

```
    public static void main(String[] args) throws Exception {

        Configuration conf = new Configuration();

        Job job = Job.getInstance(conf, "Word Count");

        job.setJarByClass(WordCount.class);

        job.setMapperClass(TokenizerMapper.class);
        job.setReducerClass(IntSumReducer.class);

        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(IntWritable.class);

        // Input and Output Path
        FileInputFormat.addInputPath(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));

        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}
```

🔷 1. BIG PICTURE: WHAT HADOOP DOES

Hadoop solves one problem:

👉 “How to process very large data (GBs/TBs) efficiently?”

✔ Solution:

- **Store data across multiple machines (HDFS)**
- **Process data in parallel (MapReduce)**

🔷 2. HADOOP ARCHITECTURE (SIMPLIFIED)

🔹 HDFS (Storage Layer)

- **NameNode** → Master (stores metadata)
- **DataNode** → Workers (store actual data)

🔹 YARN (Processing Layer)

- **ResourceManager** → Assigns resources
- **NodeManager** → Runs tasks on machines

🔷 3. WHAT HAPPENS WHEN YOU RUN WORDCOUNT

hadoop jar WordCount.jar WordCount /input /output

Now internally 👇

🔷 STEP 1: FILE STORAGE IN HDFS

Example:

Your file:

hello world hello hadoop

big data hadoop world

🔹 File Splitting

Hadoop divides file into blocks (default ~128MB)

👉 Example:

Block 1 → hello world hello

Block 2 → hadoop big data

Block 3 → hadoop world

🔹 Block Distribution

Each block is stored on different DataNodes:

DataNode1 → Block 1

DataNode2 → Block 2

DataNode3 → Block 3

👉 Also replicated (default = 3 copies)

✔ This gives:

- Fault tolerance
- Parallel processing

🔷 STEP 2: JOB SUBMISSION

When you run:

hadoop jar WordCount.jar ...

👉 Internally:

1. Job goes to **ResourceManager**
2. ResourceManager creates a **Job**
3. Splits job into **Map Tasks**

🔷 STEP 3: MAP PHASE (PARALLEL EXECUTION)

Each block → one Mapper

🔹 Example Execution

Mapper 1 (Block 1)

hello world hello

Output:

hello → 1

world → 1

hello → 1

Mapper 2 (Block 2)

hadoop big data

Output:

hadoop → 1

big → 1

data → 1

Mapper 3 (Block 3)

hadoop world

Output:

hadoop → 1

world → 1

👉 IMPORTANT:
✔ Mappers run **in parallel**
✔ Each works independently

🔷 STEP 4: SHUFFLE & SORT (MOST IMPORTANT 🔥)

👉 Hadoop automatically performs this step

🔹 What happens?

All mapper outputs are:

1. **Grouped by key**
2. **Sorted**

Before Shuffle:

hello → 1

world → 1

hello → 1

hadoop → 1

big → 1

data → 1

hadoop → 1

world → 1

After Shuffle:

big → [1]

data → [1]

hadoop → [1,1]

hello → [1,1]

world → [1,1]

👉 This step:

- Transfers data across nodes
- Groups same words together

✔ This is why Hadoop is powerful

🔷 STEP 5: REDUCE PHASE

Reducer processes grouped data:

Example:

hello → [1,1]

Reducer:

sum = 1 + 1 = 2

Final Output:

big → 1

data → 1

hadoop → 2

hello → 2

world → 2

🔷 STEP 6: OUTPUT STORAGE

👉 Results stored in HDFS:

/output/part-r-00000

✔ Each reducer creates one output file

🔷 4. HOW PARALLEL COMPUTATION HAPPENS

🔹 Key Idea: Data Locality

👉 Instead of moving data → Hadoop moves computation

✔ Mapper runs **on the same machine where data exists**

Example:

DataNode1 → Block1 → Mapper1 runs here

DataNode2 → Block2 → Mapper2 runs here

✔ Saves:

- Network cost
- Time

🔷 5. FAULT TOLERANCE (VERY IMPORTANT)

👉 If a node fails:

✔ Hadoop re-runs task on another node
✔ Uses replicated data

🔷 6. COMPLETE FLOW (SHORT SUMMARY)

1. File stored in HDFS (split into blocks)
2. Job submitted
3. Map tasks run in parallel
4. Shuffle groups data
5. Reduce computes final result
6. Output saved in HDFS

🔷 7. HOW YOUR CODE FITS INTO THIS

🔹 Mapper (Your Code)

```
context.write(word, one);
```

👉 Generates (word,1)

🔹 Reducer (Your Code)

```
sum += val.get();
```

👉 Adds values

🔷 8. IMPORTANT TERMS (FOR VIVA)

Term	Meaning
HDFS	Distributed storage system
Block	Small chunk of file
Mapper	Processes input
Reducer	Aggregates results

Shuffle	Groups keys
Data Locality	Compute near data